agyaaadssdsdsdsdsd# 📋 Control de Fases y Plan Detallado de Tareas

> **Componente:** Backend de Telemetría (Core Relay, Adquisición MMF, WebSocket y Persistencia)
> **Arquitectura:** C# .NET Clean Architecture Multicapa con `src/`
> **Propósito del Documento:** Matriz de seguimiento operativo, control de ejecución de entregables y base para estimación de esfuerzo en horas por tarea.

📊 Resumen Ejecutivo del Estado de Fases

Fase	Denominación Técnica	Estado	Tareas Totales	Tareas Completadas	Horas Estimadas
0	Especificación Técnica y Documentación Conceptual	**Completada**	5	5	**12.0 hs**
1	Andamiaje de Solución .NET, Contratos DTO y Mock Engine	**Pendiente**	11	0	**19.0 hs**
2	Adaptador Win32 Memory-Mapped Files (Assetto Corsa)	**Pendiente**	8	0	**15.0 hs**
3	Scripts de Infraestructura y Automatización Enlace USB	**Pendiente**	7	0	**8.0 hs**
4	Persistencia SQLite (Nivel 1 - Edge) y Repositorios	**Pendiente**	9	0	**17.0 hs**
5	API REST, Controladores y Middleware de Resiliencia	**Pendiente**	8	0	**14.0 hs**
6	Sincronización Asíncrona (Nivel 2 - Central PostgreSQL)	**Pendiente**	5	0	**8.5 hs**
7	Modo Demo Continuo, Suite de Testing y Cierre	**Pendiente**	5	0	**12.0 hs**
TOTAL	**Esfuerzo Global de Ingeniería (Sprint de 3 Semanas)**		**58**	**5**	**105.5 hs**

🛠️ Desglose Granular de Fases y Tareas

🟢 Fase 0: Especificación Técnica y Documentación Conceptual

Objetivo: Análisis integral de requerimientos, investigación de bajo nivel de memoria compartida Win32, diseño de arquitectura de software y elaboración de la documentación técnica.

ID	Tarea Concreta	Entregable Verificable	Estado	Horas
T-0.1	Investigación y análisis de las estructuras Win32 MMF (`acpmf_physics` , `acpmf_graphic` ,	Mapeo de campos, offsets y frecuencias de refresco documentados.	`Completada`	2.0 hs

ID	Tarea Concreta	Entregable Verificable	Estado	Horas
	`acpmf_static`) de Kunos Simulazioni.			
T-0.2	Especificación del protocolo de comunicación WebSocket a 60 Hz y contratos JSON (`FRAME` , `LAP_COMPLETED`).	Contrato de datos normalizado y documentado.	`Completada`	2.0 hs
T-0.3	Diseño de la arquitectura de persistencia en dos niveles (SQLite local in-process vs. PostgreSQL central para histórico y usuarios).	Diagrama de arquitectura y justificación técnica en documentación.	`Completada`	2.0 hs
T-0.4	Definición y delimitación formal de responsabilidades entre los roles de desarrollo (Dev 1: Backend/Host vs. Dev 2: Display Android vs. Dev 3: Web Boxes).	Matriz de responsabilidades plasmada en documentación.	`Completada`	2.0 hs
T-0.5	Elaboración y redacción de la documentación técnica oficial del backend bajo estándar Clean Architecture.	Documento `backend/DOCUMENTACION_TECNICA.md` consolidado.	`Completada`	4.0 hs

🟡 Fase 1: Andamiaje de Solución .NET, Contratos DTO y Mock Telemetry Engine

Objetivo: Crear la solución multicapa en .NET, implementar el generador de telemetría sintética a 60 Hz y montar el servidor WebSocket para desbloquear el desarrollo de las aplicaciones cliente.

ID	Tarea Concreta	Entregable Verificable	Estado	Horas
T-1.1	Creación del archivo de solución `Telemetria.sln` , los proyectos `.csproj` independientes bajo `src/` (`Dominio` , `Aplicacion` , `Infraestructura` , `Api`) y archivos `appsettings.json` iniciales.	Estructura de proyectos creada y compilable mediante `dotnet build` .	`Pendiente`	1.5 hs
T-1.2	Configuración de referencias unidireccionales entre proyectos respetando Clean Architecture y activación de `<Nullable>enable</Nullable>` .	Referencias enlazadas sin dependencias circulares ni violaciones de capas.	`Pendiente`	1.0 hs
T-1.3	Creación de la interfaz fundamental `ILectorTelemetria` en `Telemetria.Dominio/Contratos/Hardware/` .	Contrato abstracto con métodos `EstaConectado` , `ObtenerCuadroActual` , etc.	`Pendiente`	1.0 hs
T-1.4	Definición de DTOs en `Telemetria.Aplicacion/Dtos/Responses/` : `PaqueteTelemetriaDto` , `EventoVueltaCompletadaDto` y submodelos con atributos `[JsonPropertyName]` explícitos para interoperabilidad unívoca con Dev 2 y Dev 3.	DTOs fuertemente tipados con nombres en español y serialización camelCase.	`Pendiente`	2.0 hs
T-1.5	Implementación de utilitarios de dominio: método de normalización de marchas `NormalizarMarcha` (mapeo $0 \rightarrow -1, 1 \rightarrow 0, 2 \rightarrow 1, \ldots$) y cálculo métrico $d = s \times L_{\text{circuito}}$.	Normalizador probado asegurando visualización correcta de marcha.	`Pendiente`	1.5 hs
T-1.6	Implementación del generador de código de serialización *zero-allocation* `ContextoSerializacionTelemetria` mediante Source Generators de `System.Text.Json` .	Contexto compilado eliminando la reflexión en tiempo de ejecución.	`Pendiente`	1.5 hs

ID	Tarea Concreta	Entregable Verificable	Estado	Horas
T-1.7	Implementación del servicio `GeneradorTelemetriaMock.cs` con el modelado cinemático de Monza (aceleración, frenado a $-2.8\ \mathrm{G}$, cambios de marcha y trayectoria cerrada X/Z).	Servicio operativo retornando cuadros determinísticos a 60 Hz.	`Pendiente`	3.5 hs
T-1.8	Implementación del gestor concurrente `ManejadorWebSocketTelemetria.cs` en `Telemetria.Api/WebSockets/` con canales individuales `Channel<byte[]>` y política `DropOldest`.	Gestor multicliente con distribución no bloqueante y purga de desconexiones.	`Pendiente`	2.0 hs
T-1.9	Implementación del servicio en segundo plano `ServicioTelemetriaRelay.cs` como `BackgroundService` gobernado por `PeriodicTimer` a $16.66\ \mathrm{ms}$ (60 Hz).	Bucle de emisión periódico con detección de paso por meta y reinicios.	`Pendiente`	2.0 hs
T-1.10	Configuración de `Program.cs` en `Telemetria.Api`: cultura invariante (`CultureInfo.InvariantCulture`), política CORS para Dev 3 (`:3000`), Kestrel en puerto `8080`, WebSockets y ruta `/ws/telemetry`.	Servidor Kestrel levantando en puerto `8080` con pipeline y CORS configurados.	`Pendiente`	2.0 hs
T-1.11	Verificación funcional de extremo a extremo: conexión de un cliente WebSocket de prueba y recepción de flujo continuo a 60 Hz estables.	Registro de logs confirmando 60 cuadros por segundo sin jitter.	`Pendiente`	1.0 hs

Fase 2: Adaptador Win32 Memory-Mapped Files (Assetto Corsa Real)

Objetivo: Desarrollar el adaptador de bajo nivel para leer la memoria compartida del simulador en Windows, garantizando latencia en nanosegundos y consumo de CPU despreciable.

ID	Tarea Concreta	Entregable Verificable	Estado	Horas
T-2.1	Declaración de las estructuras nativas de C++ completas con el 100% de campos de Kunos Simulazioni para asegurar la alineación binaria de `carCoordinates` (`[StructLayout(LayoutKind.Sequential, Pack = 4)]`).	`EstructuraFisicaAC` (con offsets exactos), `Grafica` y `Estatica` definidas.	`Pendiente`	2.5 hs
T-2.2	Implementación de `LectorAssettoCorsaWin32.cs` implementando `ILectorTelemetria` y gestión del ciclo de vida con `IDisposable`.	Clase con apertura segura de `Local\acpmf_...` en modo de solo lectura.	`Pendiente`	2.0 hs
T-2.3	Implementación del mecanismo de tolerancia a fallos y reconexión: captura de `FileNotFoundException` y sondeo en segundo plano cuando el juego está cerrado.	El servicio no colapsa si Assetto Corsa no está abierto y conecta al iniciar.	`Pendiente`	2.0 hs
T-2.4	Implementación de detección de cambio de auto/pista: supervisar transición de `status` de 0/1 a 2 para relectura dinámica de `acpmf_static` en caliente.	Actualización automática de `maxRpm` y circuito sin reiniciar backend.	`Pendiente`	2.0 hs
T-2.5	Mapeo y transformación de tipos blittable, arreglos fijos de 4 ruedas y cadenas anchas `wchar_t` (`CharSet.Unicode`).	Conversión correcta de bytes de memoria a tipos gestionados de C#.	`Pendiente`	2.0 hs

ID	Tarea Concreta	Entregable Verificable	Estado	Horas
T-2.6	Implementación del algoritmo de detección de lecturas inconsistentes (*Torn Reads*) mediante verificación del contador atómico `packetId`.	Validación de cuadros descartando lecturas corruptas durante la carrera.	`Pendiente`	1.5 hs
T-2.7	Configuración de inyección de dependencias para conmutación transparente entre lector real y generador mock según configuración.	Capacidad de alternar proveedor de telemetría sin tocar el Relay.	`Pendiente`	1.0 hs
T-2.8	Pruebas de captura con Assetto Corsa en ejecución: verificación de lecturas de pedales, marchas y coordenadas con medición de CPU (< 0.5%).	Diagnóstico de rendimiento validado con el juego activo.	`Pendiente`	2.0 hs

Fase 3: Scripts de Infraestructura y Automatización Enlace USB

Objetivo: Proveer las herramientas de automatización en Windows para establecer el túnel físico ADB por cable USB y scripts de lanzamiento del sistema en 1 clic.

ID	Tarea Concreta	Entregable Verificable	Estado	Horas
T-3.1	Creación del directorio `backend/scripts/` para centralizar la automatización operativa.	Directorio creado con permisos de ejecución.	`Pendiente`	0.5 hs
T-3.2	Creación del script en PowerShell `iniciar-enlace-usb.ps1` con comprobación de presencia de `adb.exe` en las variables de entorno PATH.	Validación de dependencias con mensajes de error descriptivos.	`Pendiente`	1.5 hs
T-3.3	Implementación de detección de dispositivos Android conectados con verificación de estado de depuración USB autorizada.	Script que alerta si el celular no está enchufado o falta autorización RSA.	`Pendiente`	1.0 hs
T-3.4	Ejecución automatizada de túneles de retorno con `adb reverse` en puertos `8080` (Backend API/WS) y `3000` (Frontend Web) para soporte offline total en el celular.	Puertos `8080` y `3000` de la PC accesibles desde el celular a través de `localhost`.	`Pendiente`	1.5 hs
T-3.5	Creación del lanzador general `ejecutar-backend.bat` para compilar e iniciar Kestrel mediante `dotnet run --project src/Telemetria.Api` con bypass de directivas de ejecución de PowerShell.	Inicio del servidor con un solo doble clic desde el Explorador de Windows.	`Pendiente`	1.0 hs
T-3.6	Creación del script `compilar-release.bat` para empaquetar la solución en un único binario ejecutable autocontenido para Windows x64.	Publicación en un solo archivo `.exe` sin requerir SDK en la máquina final.	`Pendiente`	1.0 hs
T-3.7	Prueba física con dispositivo Android conectado por cable USB: verificación de túnel y medición de latencia (< 2.5 ms).	Transmisión de telemetría confirmada en el navegador móvil vía USB.	`Pendiente`	1.5 hs

Fase 4: Persistencia SQLite (Nivel 1 - Edge Local) y Repositorios

Objetivo: Implementar la capa de base de datos local embebida con Entity Framework Core, configuración de modo WAL para alta concurrencia e inserción masiva transaccional.

ID	Tarea Concreta	Entregable Verificable	Estado	Horas
T-4.1	Incorporación de paquetes NuGet en `Telemetria.Infraestructura`: `Microsoft.EntityFrameworkCore.Sqlite` y herramientas de diseño.	Paquetes instalados y configurados en el `.csproj`.	`Pendiente`	1.0 hs
T-4.2	Creación de modelos de dominio en `Telemetria.Dominio/Modelos/`: `EntidadBase`, `Sesion`, `Vuelta` y `MuestraTelemetria` (todas heredando de `EntidadBase` con `CreatedAt` y `UpdatedAt` obligatorios bajo 3FN).	Entidades relacionales fuertemente tipadas cumpliendo auditoría temporal.	`Pendiente`	1.5 hs
T-4.3	Implementación de `AuditoriaInterceptor.cs` para auditar automáticamente las marcas temporales `CreatedAt` y `UpdatedAt` en cada inserción/edición.	Interceptor de EF Core asignando timestamps de forma inviolable.	`Pendiente`	1.5 hs
T-4.4	Creación de `TelemetriaLocalDbContext.cs` con resolución canónica absoluta mediante `AppContext.BaseDirectory` y configuración de pragmas de alto rendimiento (`WAL`, `synchronous = NORMAL`).	DbContext configurado con ruta determinística y directivas de bajo tiempo de E/S.	`Pendiente`	2.0 hs
T-4.5	Configuración de mapeos relacionales mediante Fluent API en `Telemetria.Infraestructura/Persistencia/Configuraciones/`, incluyendo índice compuesto `(VueltaId, DistanciaPistaMetros)`.	Índices por distancia, restricciones de clave foránea y tipos de datos SQL optimizados.	`Pendiente`	2.0 hs
T-4.6	Creación y ejecución de la migración inicial de EF Core para generar el archivo de base de datos local `telemetria_local.db`.	Archivo de base de datos generado con esquema normalizado en 3FN.	`Pendiente`	1.5 hs
T-4.7	Implementación de `RepositorioVueltasSqlite.cs` e `RepositorioSesionesSqlite.cs` cumpliendo los contratos de interfaz del dominio.	Repositorios con transacciones atómicas seguras y métodos asíncronos.	`Pendiente`	2.5 hs
T-4.8	Implementación de `ServicioGestionVueltas.cs`: orquestación con el relay, máquina de estados resiliente (reinicio `current < last`, corte `tyresOut >= 4`, out-laps) y batch insert.	Persistencia transaccional de ~4.800 muestras por vuelta en < 20 ms.	`Pendiente`	3.5 hs
T-4.9	Pruebas de estrés y concurrencia: lectura de vueltas históricas simultánea a la escritura de una vuelta activa sin bloqueos de base de datos.	Verificación de cero excepciones `SQLite busy / database is locked`.	`Pendiente`	1.5 hs

Fase 5: API REST, Controladores y Middleware de Resiliencia

Objetivo: Desarrollar los controladores REST HTTP para la plataforma web de boxes, implementando normalización por distancia, paginación y manejo centralizado de excepciones.

ID	Tarea Concreta	Entregable Verificable	Estado	Horas
T-5.1	Creación de los tipos comunes en `Telemetria.Aplicacion/Common/` : envoltorio genérico `RespuestaApi<T>` y `PaginacionRespuesta<T>` .	Estructuras de respuesta uniformes para toda la API HTTP.	`Pendiente`	1.0 hs
T-5.2	Configuración de Swagger / OpenAPI con `Swashbuckle.AspNetCore` en `Telemetria.Api` para documentación interactiva de endpoints (`/swagger`).	Interfaz web de pruebas interactiva disponible para Dev 3.	`Pendiente`	1.5 hs
T-5.3	Definición de DTOs de salida específicos (`SesionResumenDto` , `VueltaDto` , `MuestraTelemetriaDto` , `EstadoServicioDto`) y sus perfiles de mapeo.	Desacoplamiento total entre modelos relacionales y JSON de salida.	`Pendiente`	1.5 hs
T-5.4	Creación de `TelemetriaController.cs` : endpoints paginados `GET /api/telemetria/sesiones` (y alias `/api/sessions`), `GET /api/telemetria/sesiones/{id}/vueltas` y `GET /api/telemetria/vueltas/{id}/telemetria` .	Endpoints operativos retornando vectores normalizados por distancia y compatibles con Dev 3.	`Pendiente`	3.0 hs
T-5.5	Creación de `EstadoController.cs` : endpoints `GET /api/estado` (y alias `/api/status` para handshake) y `POST /api/mock/alternar` (alias `/api/mock/toggle`).	Control dinámico del estado del relay sin reiniciar el servidor.	`Pendiente`	1.5 hs
T-5.6	Implementación de endpoint de comparativa analítica `GET /api/telemetria/vueltas/comparativa` con alineación métrica de dos vueltas seleccionadas.	Respuesta JSON estructurada para graficar superposición Vuelta A vs. B.	`Pendiente`	2.5 hs
T-5.7	Implementación del middleware centralizado `ManejadorExcepcionesGlobal.cs` bajo especificación RFC 7807 (`ProblemDetails`).	Manejo uniforme de errores (400, 404, 500) con logging y trazas opacas.	`Pendiente`	2.0 hs
T-5.8	Pruebas de consumo HTTP mediante Swagger y cliente HTTP validando códigos de estado y serialización camelCase.	Respuestas REST verificadas con tiempos de respuesta < 15 ms.	`Pendiente`	1.0 hs

Fase 6: Sincronización Asíncrona (Nivel 2 - Central PostgreSQL)

Objetivo: Desarrollar el mecanismo de exportación desacoplada en segundo plano (Store and Forward) para sincronizar tandas hacia la base de datos central dedicada.

ID	Tarea Concreta	Entregable Verificable	Estado	Horas
T-6.1	Definición del contrato `ISincronizadorSesiones` en `Telemetria.Dominio/Contratos/Sincronizacion/` .	Interfaz abstracta para la exportación de tandas culminadas.	`Pendiente`	1.0 hs

ID	Tarea Concreta	Entregable Verificable	Estado	Horas
T-6.2	Implementación de `SincronizadorSesionesPostgreSql.cs` en `Telemetria.Infraestructura/Sincronizacion/` mediante cliente relacional Npgsql/Dapper.	Conector a base de datos central estructurado bajo esquema 3FN.	`Pendiente`	3.0 hs
T-6.3	Implementación del trabajador en segundo plano `ServicioSincronizacionSegundoPlano.cs` como `BackgroundService` en Kestrel con reintentos silenciosos y tolerancia offline.	Worker periódico que sondea tandas terminadas cada 60s sin saturar logs si la BD central no está disponible.	`Pendiente`	2.0 hs
T-6.4	Implementación de la lógica de actualización atómica: marcar `SincronizadoConServidorCentral = true` en SQLite tras confirmación exitosa.	Garantía de no duplicación de tandas en el servidor central.	`Pendiente`	1.0 hs
T-6.5	Pruebas de resiliencia de red: simulación de desconexión del servidor central, validando que las sesiones se preserven en SQLite y se exporten al reconectar.	Mecanismo *Store and Forward* validado sin pérdidas de información.	`Pendiente`	1.5 hs

Fase 7: Modo Demo Autónomo, Suite de Testing y Cierre

Objetivo: Desarrollar el modo de demostración continua pregrabada, suite de pruebas automatizadas y compilación final de producción.

ID	Tarea Concreta	Entregable Verificable	Estado	Horas
T-7.1	Grabación y empaquetado de una vuelta real completa de Assetto Corsa como recurso embebido (`EmbeddedResource` en `vuelta-referencia-monza.json`).	Archivo de datos reales pregrabados compilado dentro del binario.	`Pendiente`	1.5 hs
T-7.2	Implementación del modo de reproducción demo continua en bucle infinito a 60 FPS sin necesidad de simulador abierto.	Parámetro de inicio `--demo` reproduciendo telemetría idéntica a la real.	`Pendiente`	2.5 hs
T-7.3	Creación de pruebas unitarias con xUnit y Moq en `Telemetria.Dominio.Tests/` y `Telemetria.Aplicacion.Tests/` .	Cobertura de pruebas sobre el motor mock, orquestador y detección de vuelta.	`Pendiente`	3.5 hs
T-7.4	Creación de pruebas de integración en `Telemetria.Api.IntegrationTests/` utilizando `WebApplicationFactory` .	Validación automatizada de endpoints REST y negociación de WebSockets.	`Pendiente`	2.5 hs
T-7.5	Compilación final en modo Release, verificación de consumo de memoria plano sin fugas y actualización de documentación de cierre.	Binario listo para despliegue y registro exhaustivo completado.	`Pendiente`	2.0 hs